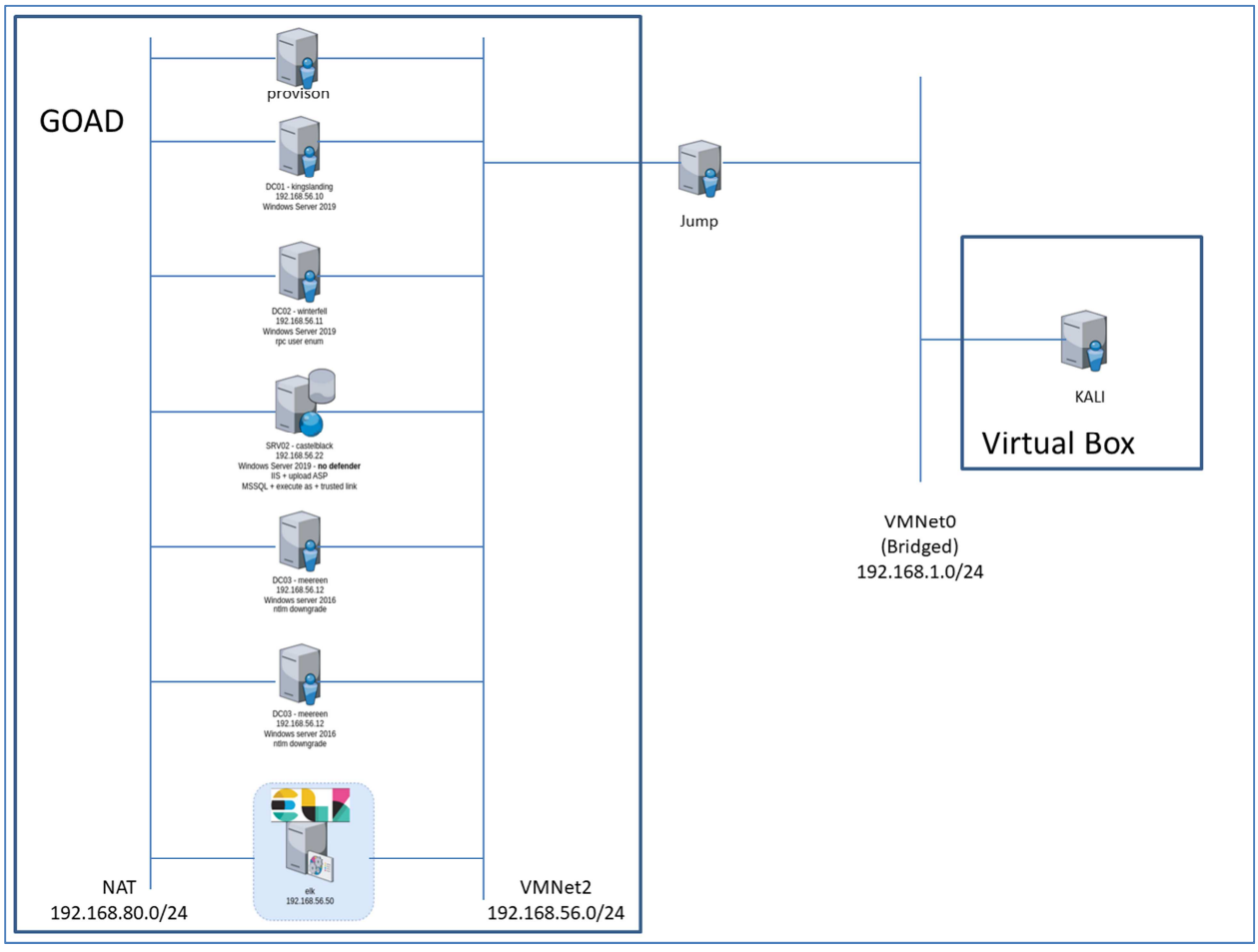
GOAD Lab Access via a Jump Box

JJKirn 1/5/26

Intro:

This document shows a method to add a **“Jump” box VM** to an existing operational **GOAD LAB** running on a **VMware Workstation Pro** instance on a **Windows 11 host** to provide remote access. The remote access can even be via a **Kali VM** running on **Virtual Box** on a **separate Windows host** system as long as there is a **shared bridged network**.

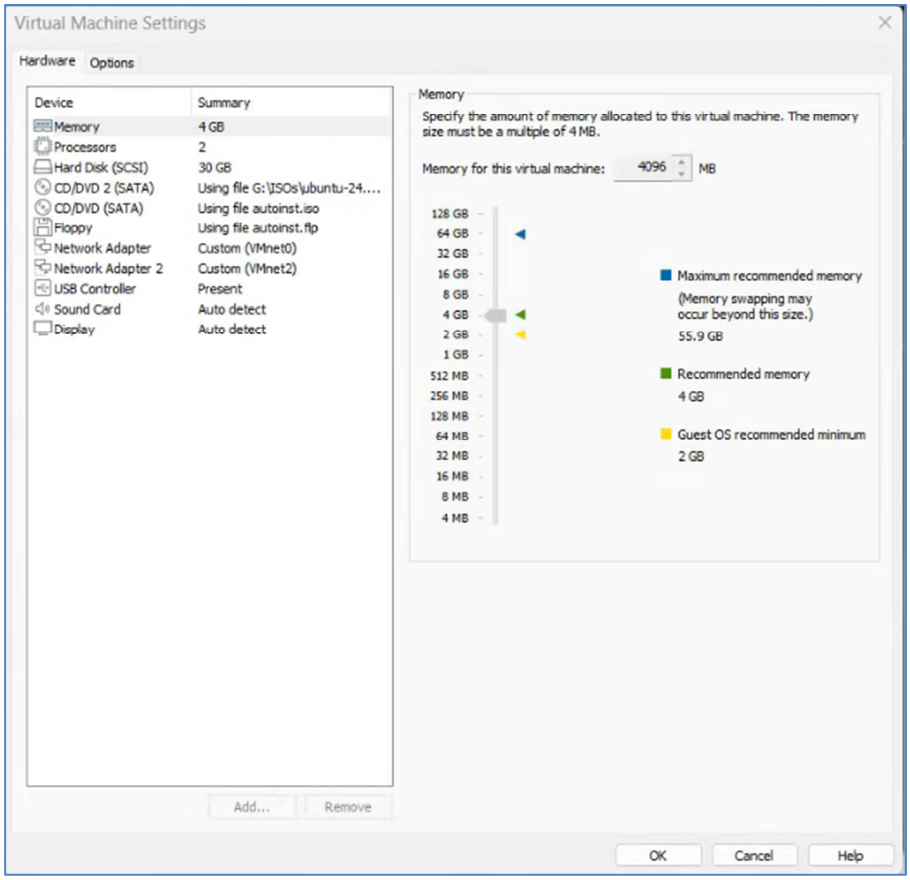


GOAD LAB Network with **Jump Box** and **Remote Windows host Virtualbox Kali VM**

Start with the “Jump” box VM:

Step 1: Create the “Jump” box VM on the same VMware Workstation that is running the GOAD lab.

I chose **Ubuntu 24.04 Desktop** for the VM. Here is the configuration I used:



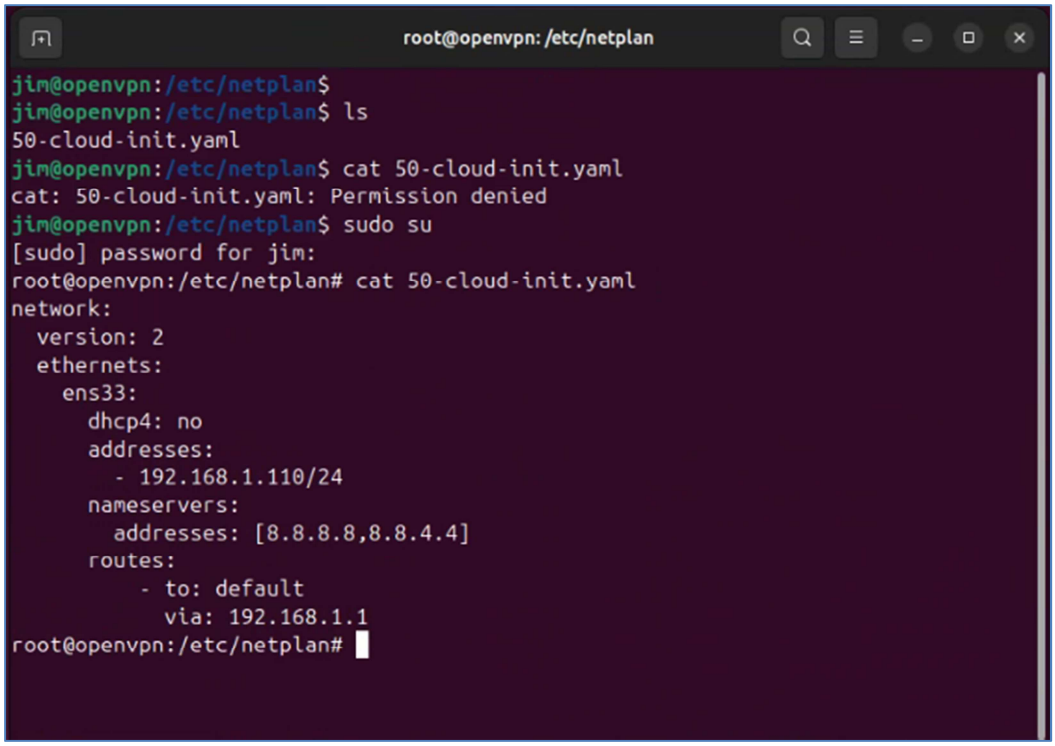
Once you power it up, you will need to answer the usual questions about username and password, hostname, timezone, etc… to be able to create the VM.

First step is to do the usual update/upgrade. Open a terminal window and enter:

```
$ sudo apt upgrade
$ sudo apt upgrade
```

Step 2: Configure the “Jump” box for static IP

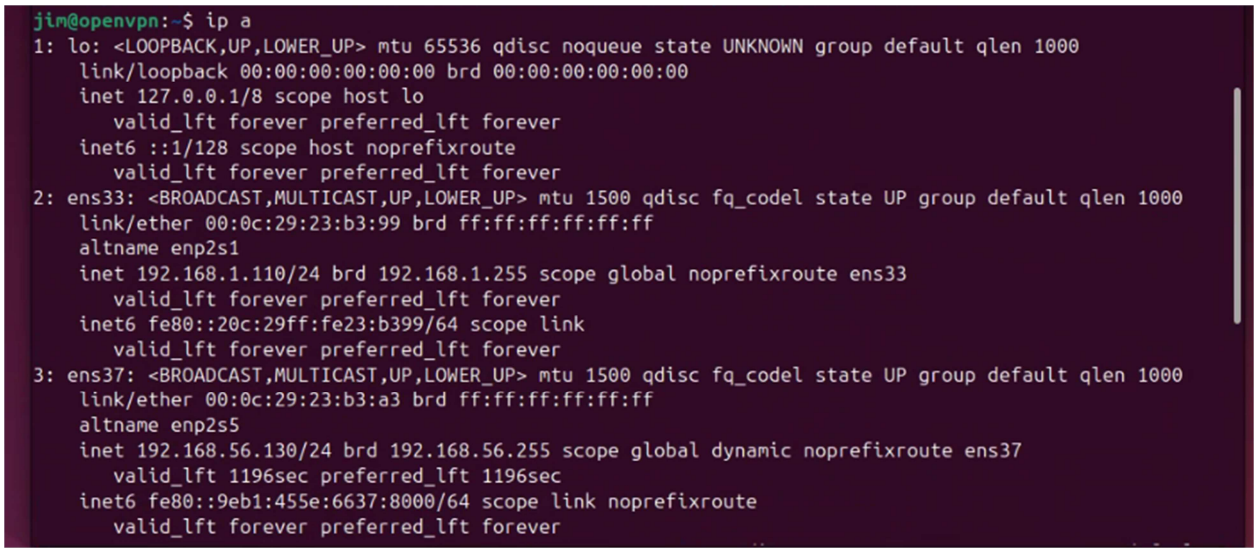
As root user, edit **`/etc/netplan/50-cloud-init.yml`** file as follows:



As you can see, I chose **192.168.1.110** as the **static IP** for the **"Jump" box**. To make these changes effective run:

```
$ sudo netplan apply
```

Verify ip address is active by running the **`ip a`** command:



Step 3: Make sure ssh server is running:

Run the following command to install openssh server:

```
$ sudo apt install openssh-server
```

Next Steps are on the remote Windows host Kali VM

Step 1: Start by installing a Kali VM (using VirtualBox in my case)

I assume you know how to download a **Kali ISO** and install it.

Remember we are doing these steps on a **remote Windows host machine** using **VirtualBox**.

Kali settings:

Name: kali
Base Memory: 4GB
Displays: 1
Storage: 120GB
Network Adapter 1: Bridged

Step 2: Set a static IP

I chose to assign a **static IP to the VM (192.168.1.200)**.

As root user edit /etc/network/interfaces file:

```
┌──(root㉿kali)-[/etc/network]
└─# cat interfaces
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
    address 192.168.1.200
    netmask 255.255.255.0
    gateway 192.168.1.1
    #dns-nameserver 8.8.8.8
```

Restart networking to make changes effective:

```
┌──(jim㉿kali)-[~/Desktop]
└─$ sudo systemctl restart networking
```

Verify Changes worked by running ip a:

```
┌──(jim㉿kali)-[~/Desktop]
└─$ ip a
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state
UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
fq_codel state UP group default qlen 1000
    link/ether 08:00:27:8b:7c:42 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.200/24 brd 192.168.1.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe8b:7c42/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

Step 3: Make sure ssh server is running:

Run the following command to install openssh server:

```
$ sudo apt install openssh-server
```

Step 4: Create a SSH config file at ~/.ssh/config:

```
┌──(jim㉿kali)-[~/.ssh]
└─$ cat config
Host jumphost
    HostName 192.168.1.110
    User jim

Host 192.168.56.*
    ProxyCommand ssh -W %h:%p jumphost
    User vagrant
```

Step 5: Configure proxychains4

```
# Install proxychains
sudo apt install proxychains4

# Edit /etc/proxychains4.conf:
# make sure you comment out the following line
#socks4 127.0.0.1 9050

# Then add this at end of file
socks5 127.0.0.1 8080
```

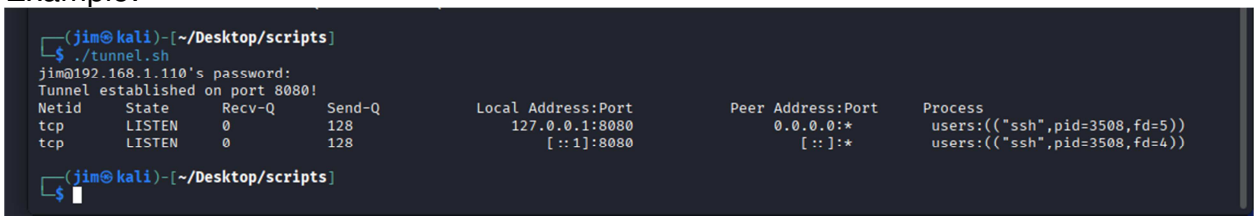
You are now ready to access the **GOAD Lab** through the use of **proxychains4**. Below are some example scripts that will help simply use of various tools.

Script Examples:

1. **Create script for creating the ssh tunnel (tunnel.sh):**

```
┌──(jim㉿kali)-[~/Desktop]
└─$ cat tunnel.sh
#!/bin/bash
ssh -D 8080 -f -N -o ServerAliveInterval=60 -o
ServerAliveCountMax=3 jim@192.168.1.110
echo "Tunnel established on port 8080!"
ss -tulnp
```

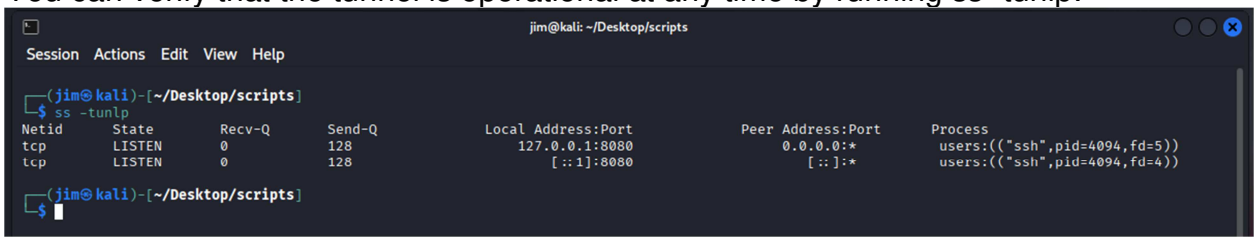
Example:



Note: The above script must be run at least once before running any of the following scripts or any proxychain4 commands.

For the following scripts you need to add one of the five server names (kingslanding, castelblack, …) for them to run correctly.

You can verify that the tunnel is operational at any time by running ss -tunlp:

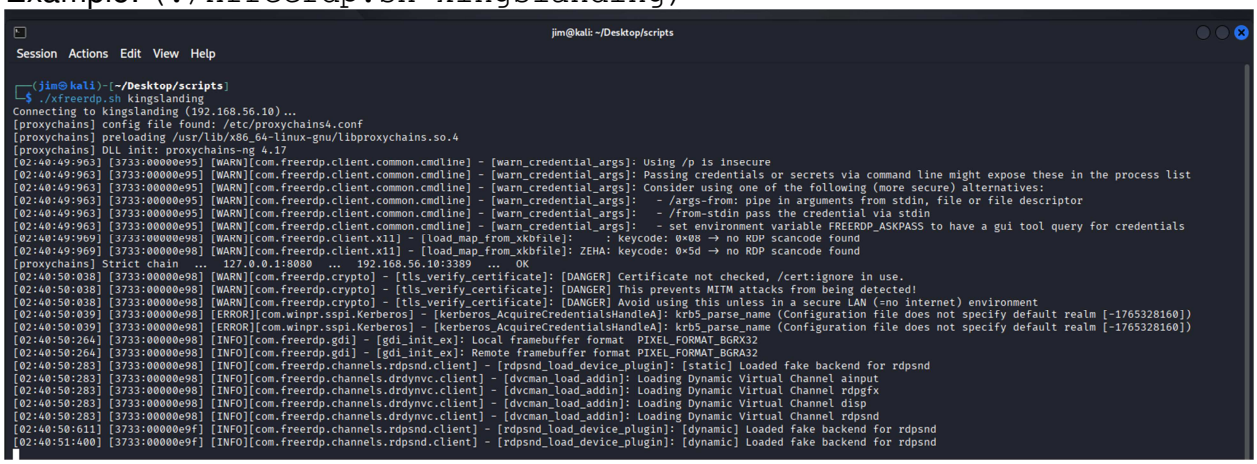


2. **This script launches a xfreerdp session to the selected server:**

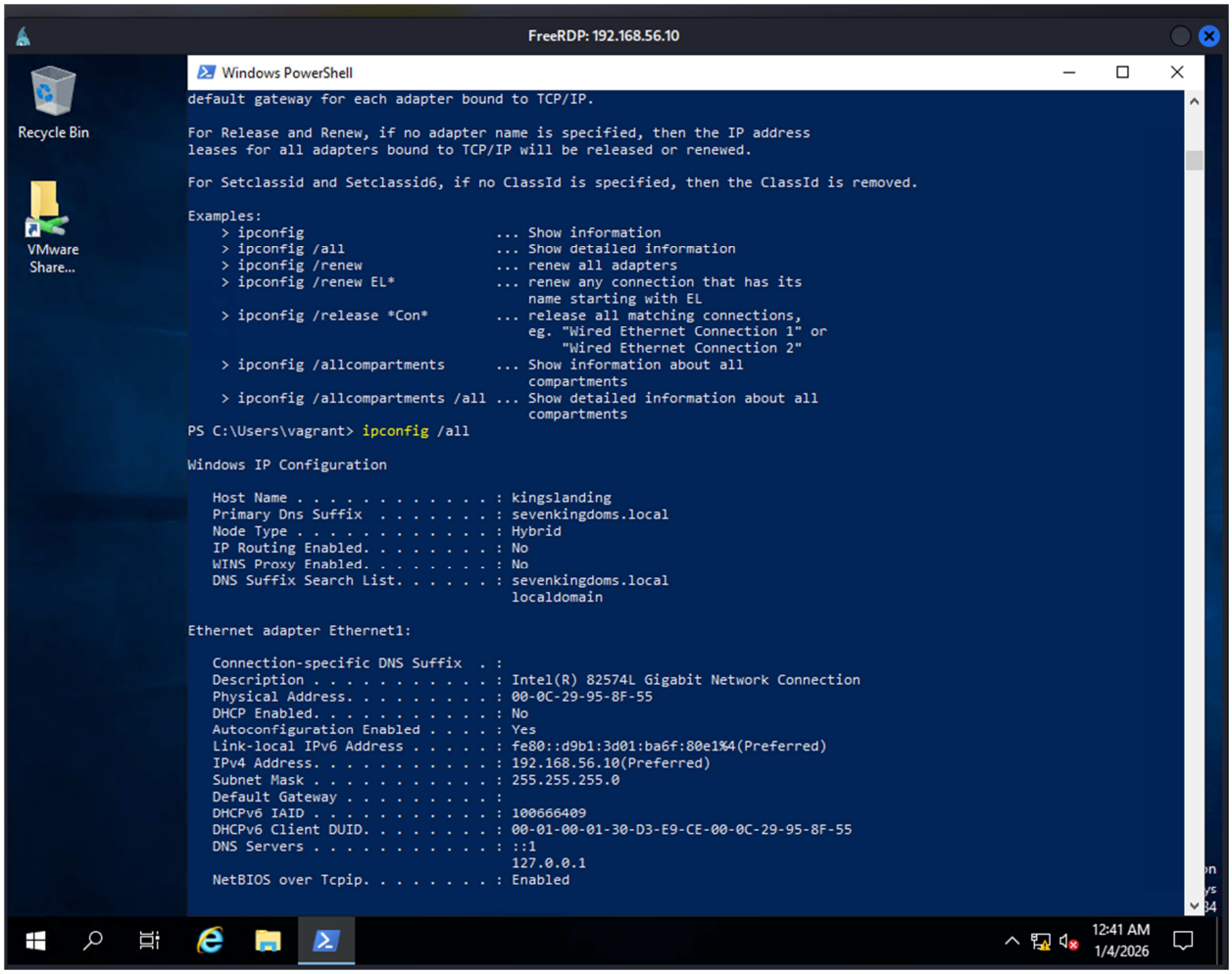
```
┌──(jim㉿kali)-[~/Desktop/scripts]
└─$ cat xfreerdp.sh
#!/bin/bash
# Resolve hostname from /etc/hosts and connect via proxychains

HOST=$1
IP=$(grep -w "$HOST" /etc/hosts | awk '{print $1}')

if [ -z "$IP" ]; then
    echo "Host $HOST not found in /etc/hosts"
    exit 1
```

```
fi

echo "Connecting to $HOST ($IP)..."
proxychains4 xfreerdp3 /compression /cert:ignore +auto-reconnect
/v:$IP /u:vagrant /p:vagrant +clipboard
```

Example: (`./xfreerdp.sh kingslanding`)



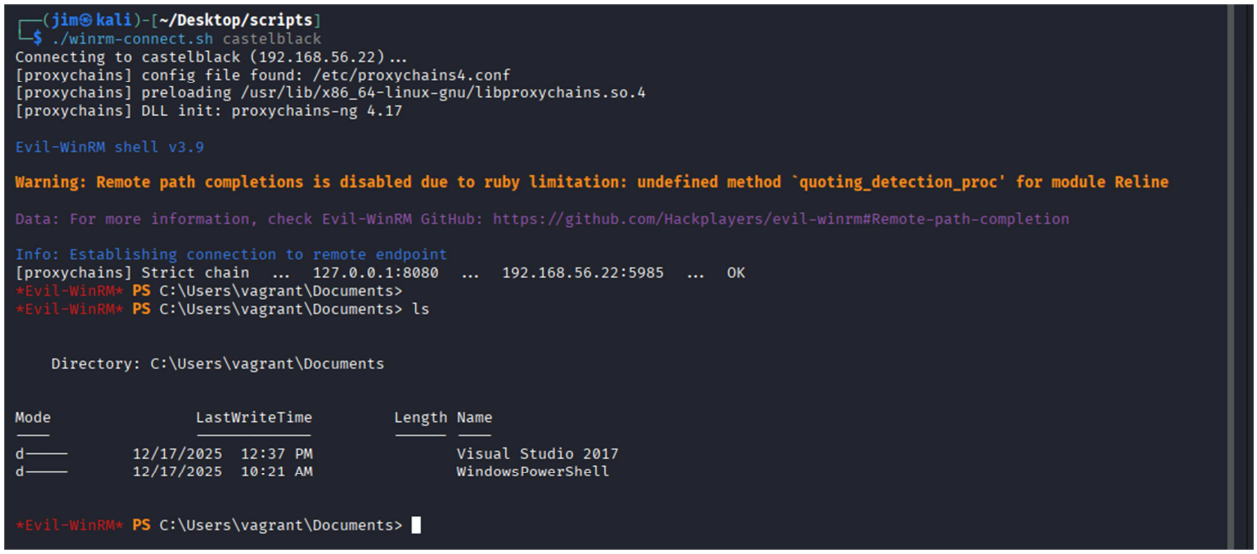
And the xfreerdp window appears as:



3. **This script launches an evil-winrm session to the selected server:**

```
┌──(jim㉿kali)-[~/Desktop/scripts]
└─$ cat winrm-connect.sh
#!/bin/bash
# Resolve hostname from /etc/hosts and connect via proxychains

HOST=$1
IP=$(grep -w "$HOST" /etc/hosts | awk '{print $1}')

if [ -z "$IP" ]; then
    echo "Host $HOST not found in /etc/hosts"
    exit 1
fi

echo "Connecting to $HOST ($IP)..."
proxychains4 evil-winrm -i $IP -u vagrant -p vagrant
```

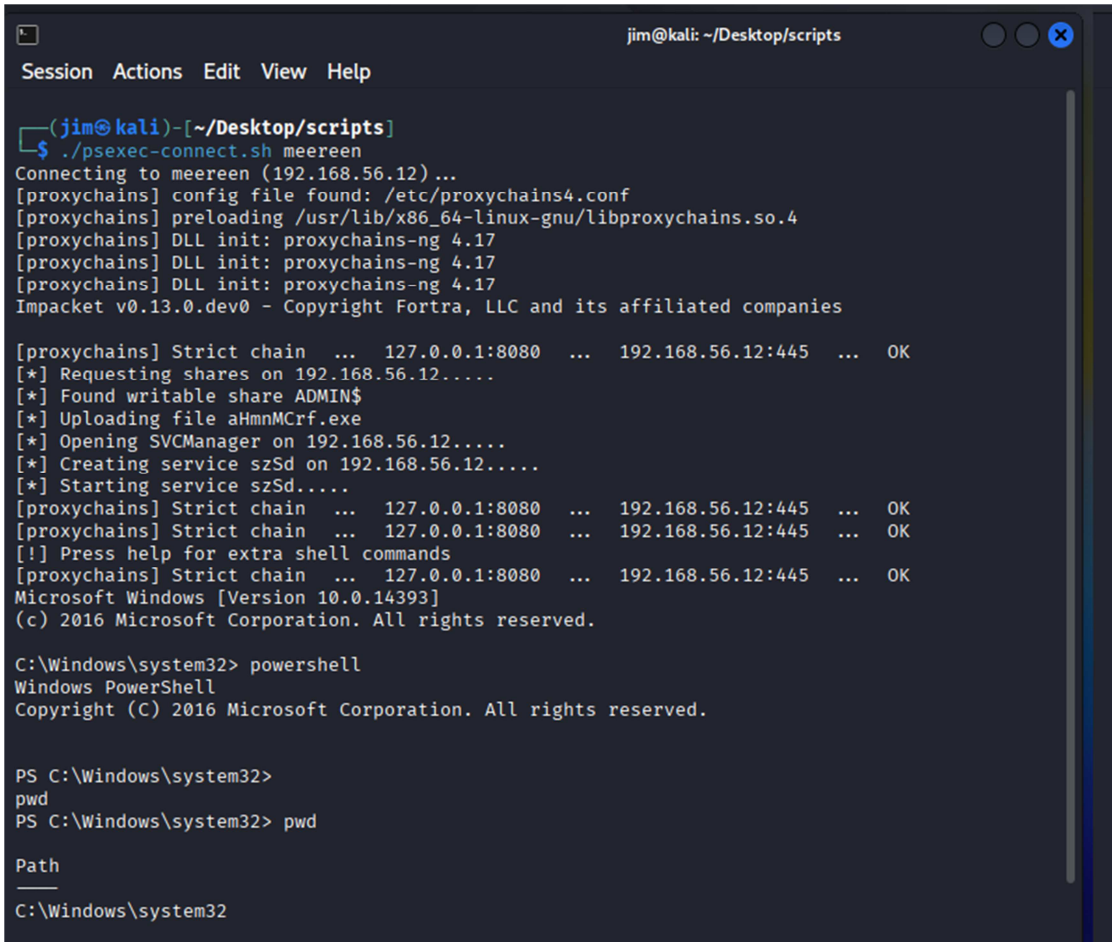
Example: (./winrm-connect.sh castelblack)



4. **This script launches a PSExec session to the selected server:**

```
┌──(jim㉿kali)-[~/Desktop/scripts]
└─$ cat psexec.sh
#!/bin/bash
# Resolve hostname from /etc/hosts and connect via proxychains

HOST=$1
IP=$(grep -w "$HOST" /etc/hosts | awk '{print $1}')

if [ -z "$IP" ]; then
    echo "Host $HOST not found in /etc/hosts"
    exit 1
fi

echo "Connecting to $HOST ($IP)..."
proxychains4 impacket-psexec vagrant:vagrant@$IP
```

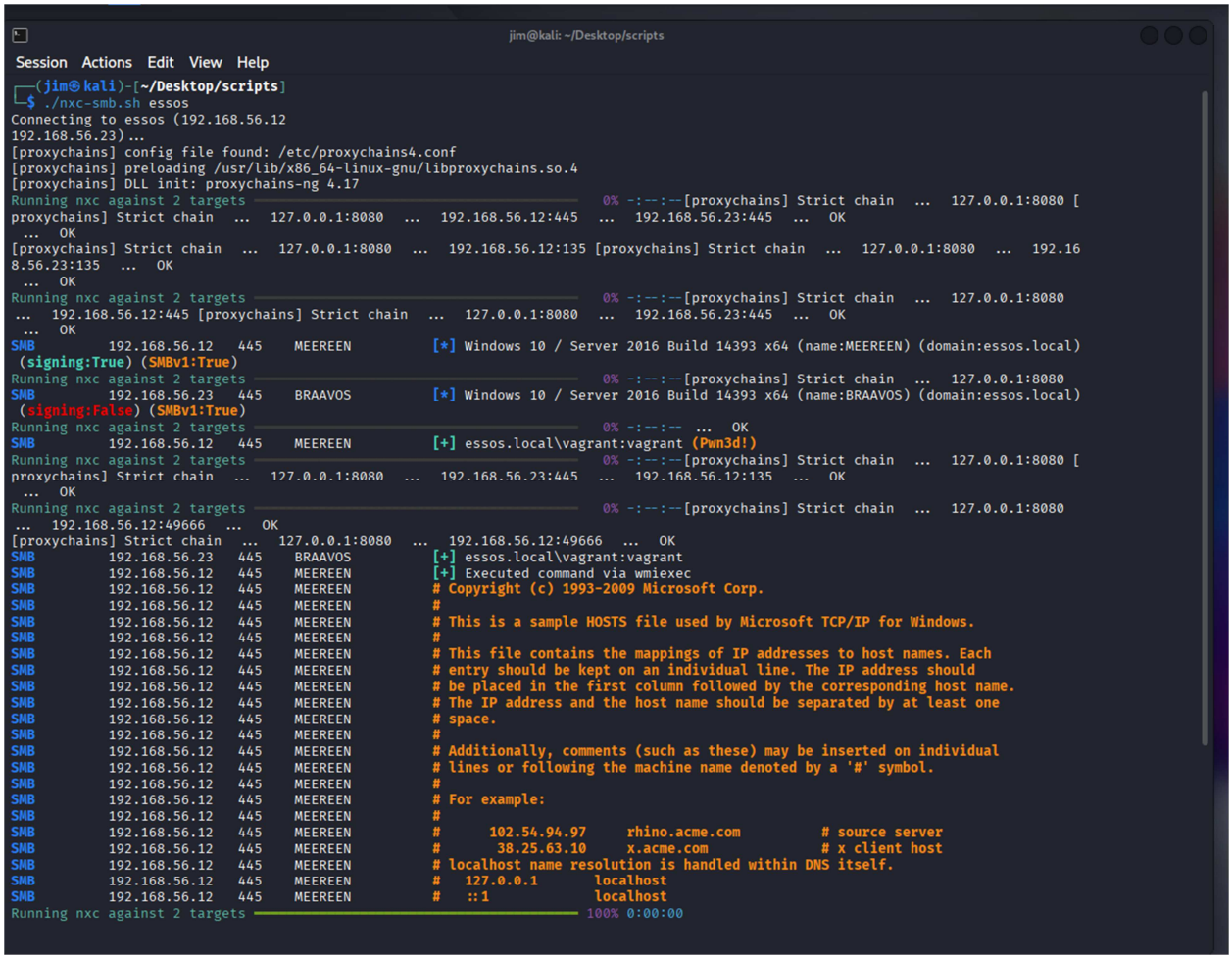
Example: (./psexec-connect.sh meereen)



5. **This script launches a nxc smb session to the selected server:**

```
┌──(jim㉿kali)-[~/Desktop/scripts]
└─$ cat nxc-smb.sh
#!/bin/bash
# Resolve hostname from /etc/hosts and connect via proxychains

HOST=$1
IP=$(grep -w "$HOST" /etc/hosts | awk '{print $1}')

if [ -z "$IP" ]; then
    echo "Host $HOST not found in /etc/hosts"
    exit 1
fi

echo "Connecting to $HOST ($IP)..."
proxychains4 nxc smb -u vagrant -p vagrant -x "type
C:\Windows\System32\drivers\etc\hosts"
```

Example: (./nxc-smb.sh meereen)



The above scripts are just some examples of how you can automate using proxychains4 to perform usual pentesting commands on a remote system through the use of a **Jump box VM**.

Obviously you can use **proxychains4** directly with any typical Kali attack tool.

Enjoy!